Streamlined - Wind Tunnel Data Reduction Package

Author: C. Fagley
Version: 1.2.4

Executive Summary

Streamlined is a Python-based wind tunnel data reduction system that replaces legacy MATLAB processing workflows. The package processes raw force balance voltage measurements from TDMS data files, applies multi-order calibrations, transforms forces through body and wind reference frames with proper tare subtraction, and computes aerodynamic coefficients. All operations are performed on full time-varying vectors; steady-state values are the temporal mean of fully-reduced coefficient time histories.

Key Features:

- Complete MATLAB-to-Python migration with validated parity
- Modern PyQt6-based GUI with interactive pyqtgraph visualization
- Compressible isentropic tunnel condition calculations with Sutherland viscosity
- Force and moment balance configurations with automatic channel mapping
- Multi-geometry support with per-case geometry assignment
- Unified export to CSV, Excel, HDF5, and MAT formats
- Interactive plot export with customizable themes, legend, and styling
- LabVIEW-callable balance calibration interface
- Support for multiple wind tunnel facilities (SWT, LSWT, TST)

Table of Contents

14. Graphical User Interface
15. Export Capabilities
16. Technical Specifications

Data Reduction Pipeline (Step by Step)

The complete processing flow from raw voltage to aerodynamic coefficient. Every operation prior to Stage 9 operates on the **full time-varying vectors** (every DAQ sample). Averaging to a single steady-state value per test point occurs only at the final stage.

```
TDMS Files (raw voltages)
    |
    v
[Stage 1] Read & Resample to common time base
    |
    v
[Stage 2] Load balance calibration (.vol) --> polynomial coefficients
    |
    v
[Stage 3] Apply calibration: V_norm = V_raw / V_excitation
          Form polynomial terms, multiply by calibration matrix
          --> element forces [N1, N2, Y1, Y2, Axial, Roll] (time-series)
    |
    +------ Air-ON path ------+------ Air-OFF path ------+
    |                         |                          |
    v                         v                          |
[Stage 4] BRF forces       [Stage 4] BRF forces          |
    (using Air-ON alpha)      (using Air-OFF alpha)       |
    |                         |                          |
    v                         v                          |
[Stage 5] WRF forces       [Stage 5] WRF forces          |
    (using Air-ON alpha/beta) (using Air-OFF alpha/beta)  |
    |                         |                          |
    +----------+--------------+                          |
               |                                         |
               v                                         |
[Stage 6] Tare subtraction in WRF (DC removal):           |
          Lift_aero = Lift_on - mean(Lift_off)            |
          Drag_aero = Drag_on - mean(Drag_off) (etc.)     |
               |                                        |
               v                                        |
[Stage 7] Tunnel conditions (compressible isentropic)     |
          from Air-ON P0, dP, T0                          |
          --> Q, Mach, rho, U_inf, Re (time-series)       |
               |                                        |
               v                                        |
[Stage 8] Aerodynamic coefficients:                       |
          CL(t) = Lift_aero(t) / (Q(t) * S)              |
          (every sample is a fully-reduced coefficient)   |
               |                                        |
               v                                        |
[Stage 9] Steady-state: mean(CL(t)) --> single CL value  |
          std(CL(t)) --> CL_std (flow unsteadiness)       |
```

Critical design point: Tare subtraction happens in the **Wind Reference Frame**, not the Body Reference Frame. Air-on and air-off data are each transformed through BRF and WRF using **their own** alpha/beta angles before subtraction. This ensures correct weight tare removal when the model is at any attitude.

Stage 1: Data Loading (TDMS)

Source: `data_io.py` -> `read_tdms_file()`

Raw data is acquired as TDMS (Technical Data Management Streaming) files from National Instruments DAQ systems.

Filename Convention:

```
[AirState]_[Configuration]_Alpha_[value]_Beta_[value].tdms

Examples:
  AirOn_F16check_Alpha_-2.0_Beta_0.0.tdms
  AirOff_F16check_Alpha_0.0_Beta_0.0.tdms
```

Processing Steps:

1. **Open TDMS file** using `nptdms` library
2. **Extract all channels** from all groups (balance voltages, pressures, temperatures, angles)
3. **Build time vectors** from channel properties (`wf_increment`, `wf_samples`):

   ```
   time = [0, dt, 2*dt, ..., (N-1)*dt]
   ```

4. **Find the highest-rate channel** (smallest `dt`) as the reference time base
5. **Resample all channels** to the reference time base via cubic interpolation (`scipy.interpolate.interp1d`) for channels with different sample rates
6. **Extract Alpha/Beta** from (in priority order):
   - DAQ channels (time-varying encoder signals)
   - TDMS group properties (single values)
   - Filename parsing via regex: `Alpha[_\s]*(-?\d+\.?\d*)`

Output: Dictionary of numpy arrays, one per channel, all at the same sample rate:

```
{
  'N1': array([...]),     # Normal force gauge 1 voltage
  'N2': array([...]),     # Normal force gauge 2 voltage
  'Y1': array([...]),     # Side force gauge 1 voltage
  'Y2': array([...]),     # Side force gauge 2 voltage
  'Axial': array([...]),  # Axial force gauge voltage
  'Roll': array([...]),   # Roll moment gauge voltage
  'Excitation': array([...]),  # Bridge excitation voltage
  'Pdiff': array([...]),  # Differential pressure transducer
  'Ptot': array([...]),   # Total pressure transducer
  'Temp': array([...]),   # Temperature sensor
  'Alpha': array([...]),  # Angle of attack
```

```
  'Beta': array([...]),   # Sideslip angle
  'Time': array([...]),   # Common time vector
}
```

Stage 2: Balance Calibration

Source: `calibration.py` -> `read_vol_file()`, `calc_coeffs()`

The force balance is calibrated by applying known loads and recording the resulting voltages. The `.vol` file stores this calibration data.

Reading the .vol File

The `.vol` file contains:

- **Balance Description:** type, serial number, outer diameter
- **Maximum Balance Loads:** load limits per channel with units
- **Distances:** moment arm distances `dx1`, `dx2`, `dy1`, `dy2` (distance from balance center to each gauge)
- **Calibration Curves:** for each of the 6 channels (N1, N2, Y1, Y2, Axial, Roll), both positive and negative loading directions

Each calibration curve section contains:

```
[N1pos]
NumberOfLoads --> 5
Force, V1, V2, V3, V4, V5, V6, Vexcite
0.0,  0.001, -0.002, 0.000, 0.001, -0.001, 0.000, 5.002
10.0, 0.452, -0.003, 0.001, 0.002, -0.001, 0.001, 5.001
...
```

Processing the calibration data:

1. **Parse Force and Voltage matrices** from all positive and negative loading sections
2. **Zero-offset correction:** Find rows where applied force = 0, compute mean voltage offset, subtract from all voltage readings
3. **Normalize voltages** by excitation voltage: `V_norm = (V_raw - V_zero) / V_excitation`
4. **Build matrices:**
    - `Force[n_total_loads x 6]` — applied forces (negative loadings get sign-flipped)
    - `Volts[n_total_loads x 6]` — normalized voltages

Computing Calibration Coefficients

Source: `calibration.py` -> `calc_coeffs()`

Three calibration types are supported:

Type	Polynomial Terms	Matrix Shape
Linear	`[V]`	6 x 6

Type	Polynomial Terms	Matrix Shape
Quadratic	`[V, V^2]`	12 x 6
Cubic	`[V, V^2, V^3]`	18 x 6

Steps:

1. **Form polynomial terms** from the voltage matrix:

   ```
   Linear:    X = V                          (n x 6)
   Quadratic: X = [V | V.*V]                 (n x 12)
   Cubic:     X = [V | V.*V | V.*V.*V]       (n x 18)
   ```

   where `.*` denotes element-wise multiplication (each column squared/cubed independently).

2. **Least-squares solve** for calibration coefficient matrix:

   ```
   X @ Coeffs = Force
   Coeffs = lstsq(X, Force)
   ```

   Result: `Coeffs` matrix of shape `(n_terms x 6)`.

3. **Compute fit quality:**

   ```
   Force_est = X @ Coeffs
   R^2 = 1 - SS_res / SS_tot     (per channel)
   Bias = RMSE = sqrt(sum((Force - Force_est)^2) / n)    (per channel)
   ```

Stage 3: Apply Calibration to Raw Voltages

Source: `transforms.py` -> `calc_brf_forces()` (first half)

For each test point (each TDMS file), the raw voltage time-series are converted to element forces using the calibration matrix.

Steps:

1. **Normalize raw voltages** by excitation voltage (sample by sample):

   ```
   V_norm[i, :] = [N1[i], N2[i], Y1[i], Y2[i], Axial[i], Roll[i]] /
   Excitation[i]
   ```

   This produces a matrix of shape `(n_samples x 6)`.

2. **Form polynomial terms** (same order as calibration):

   ```
   Linear:    X = V_norm                                (n_samples x 6)
   Quadratic: X = [V_norm | V_norm.*V_norm]              (n_samples x 12)
   Cubic:     X = [V_norm | V_norm.*V_norm | V_norm^3]   (n_samples x 18)
   ```

3. **Multiply by calibration matrix:**

```
Elements = X @ Coeffs    (n_samples x 6)
```

Each row gives `[N1_force, N2_force, Y1_force, Y2_force, Axial_force, Roll_moment]` at that time instant.

Output: `elements` matrix — time-series of balance element forces in engineering units (typically lbf).

Stage 4: Body Reference Frame (BRF) Forces

Source: `transforms.py` -> `calc_brf_forces()` (second half)

Element forces are combined into body-axis forces and moments, with moment reference center (MRC) shift corrections.

Balance Axis System

```
6-Component Internal Balance:
  N1, N2 : Normal force gauges (pitch plane, fore/aft)
  Y1, Y2 : Side force gauges (yaw plane, fore/aft)
  Axial  : Axial force gauge
  Roll   : Rolling moment gauge

Gauge distances (from balance center):
  dx1 : distance from center to N1 gauge
  dx2 : distance from center to N2 gauge
  dy1 : distance from center to Y1 gauge
  dy2 : distance from center to Y2 gauge
```

Force Configuration (standard)

Forces are direct sums of element pairs. Moments are computed from element differences scaled by moment arms, with MRC shift corrections:

```
Fz = N1 + N2                                    (Normal force)
Fy = Y1 + Y2                                    (Side force)
Fx = Axial                                       (Axial force)

Mx = Roll - Fy * mshift_z                        (Rolling moment)
My = N1*(dx1 - mshift_x) - N2*(dx2 + mshift_x)
     - Fx * mshift_z                             (Pitching moment)
Mz = Y1*(dy1 + mshift_y) - Y2*(dy2 - mshift_y)
     - Fy * mshift_x                             (Yawing moment)
```

Where `mshift = [mshift_x, mshift_y, mshift_z]` is the MRC offset from balance center.

Moment Configuration (alternative)

For moment-type balances, the six channels are named `AftPitch, AftYaw, FwdPitch, FwdYaw, Axial, Roll`. Forces are derived from moment differences and moments are averages:

```
Fz = (AftPitch - FwdPitch) / (dx1 + dx2)
Fy = (AftYaw  - FwdYaw)  / (dy1 + dy2)
Fx = Axial

Mx = Roll - Fy * mshift_z
My = (AftPitch + FwdPitch) / 2 - Fx * mshift_z - Fz * mshift_x
Mz = (AftYaw  + FwdYaw)  / 2 + Fx * mshift_y + Fy * mshift_x
```

Note: For moment balances, if `dx1 > 2` the distances are halved (they represent full station-to-station distance rather than center-to-gauge distance).

The GUI automatically adapts element labels in the time history viewer and data table to show AftPitch/AftYaw/FwdPitch/FwdYaw when Moment config is selected, or N1/N2/Y1/Y2 for Force config. If the DAQ data uses N1/N2/Y1/Y2 channel names with a moment balance, the software maps them positionally (N1 to AftPitch, N2 to AftYaw, Y1 to FwdPitch, Y2 to FwdYaw).

Output: `BRFForces` containing `Fx, Fy, Fz, Mx, My, Mz` — all time-series vectors.

Stage 5: Wind Reference Frame (WRF) Transformation

Source: `transforms.py` -> `calc_wrf_forces()`

Body-axis forces are rotated into the wind reference frame using the angle of attack (alpha) and sideslip angle (beta). Alpha and beta may be time-varying encoder signals.

Transformation Equations

```
alpha_rad = alpha * pi / 180
beta_rad  = beta * pi / 180

Lift = Fz * cos(alpha) - Fx * sin(alpha)

Drag = Fx * cos(beta) * cos(alpha)
     - Fy * sin(beta)
     + Fz * sin(alpha) * cos(beta)

Side = Fx * sin(beta) * cos(alpha)
     + Fy * cos(beta)
     + Fz * sin(alpha) * sin(beta)

Roll  = Mx    (moments transfer directly)
Pitch = My
Yaw   = Mz
```

Important: This transformation is applied **separately** to air-on and air-off data using **their own** alpha/beta angles. Air-off data may be at a different angle than air-on (e.g., single-tare approach at alpha=0).

Output: `WRFForces` containing `Lift, Drag, Side, Roll, Pitch, Yaw` — all time-series vectors.

Stage 6: Tare Subtraction (Air-Off Removal)

Source: `transforms.py` -> `subtract_wrf_forces()`

The weight tare (air-off) is removed by subtracting the **mean** (DC component) of each air-off WRF force channel from the corresponding air-on time-series. This preserves the time-varying aerodynamic content of the air-on signal while removing only the static weight bias.

```
Lift_aero(t) = Lift_on(t) - mean(Lift_off)
Drag_aero(t) = Drag_on(t) - mean(Drag_off)
Side_aero(t) = Side_on(t) - mean(Side_off)
Roll_aero(t) = Roll_on(t) - mean(Roll_off)
Pitch_aero(t) = Pitch_on(t) - mean(Pitch_off)
Yaw_aero(t)  = Yaw_on(t)  - mean(Yaw_off)
```

Because both air-on and air-off were transformed to WRF using their respective alpha/beta angles, the weight components resolve correctly regardless of model attitude. The air-off mean is a scalar per channel, so the output length matches the air-on signal exactly.

The same DC-removal approach is applied to pressure port tare subtraction:

```
P_aero_i(t) = P_on_i(t) - mean(P_off_i)
```

Output: `WRFForces` (aerodynamic) -- time-series of pure aerodynamic loads with weight tare removed.

Stage 7: Tunnel Conditions

Source: `coefficients.py` -> `calc_tunnel_conditions()`

Tunnel flow conditions are computed from the air-on pressure and temperature sensor data using **compressible isentropic relations**. All outputs are time-series vectors.

Instrumentation

Sensor	Measures	Symbol	Notes
Pdiff transducer	Differential pressure (P0 - P_static)	dP	Voltage * cal slope gives psi
Ptot transducer	Absolute total (stagnation) pressure	P0	Voltage * cal slope gives psi
Temp thermocouple	Total (stagnation) temperature	T0	In settling chamber; raw * 10 = Fahrenheit

SWT (Subsonic Wind Tunnel) Calculations

Step 1: Convert measured quantities to SI

```
dP_psi = Pdiff_raw * slope_pdiff
dP_Pa  = dP_psi * 6894.75729

P0_Pa  = Ptot_raw * slope_p0 * 6894.75729

T0_F   = Temp_raw * 10
T0_C   = (T0_F - 32) * 5 / 9
T0_K   = T0_C + 273.15
```

Where `slope_pdiff` and `slope_p0` are pressure transducer calibration slopes from the .PCF file.

Step 2: Derive static pressure

```
P_static = P0 - dP
```

(Clamped to a minimum of 1.0 Pa to prevent division by zero.)

Step 3: Isentropic core term

The key quantity linking total and static pressure through isentropic flow:

```
isentropic_term = (P0 / P_static)^((gamma-1)/gamma) - 1
```

Step 4: Compressible dynamic pressure

The compressible dynamic pressure is NOT equal to dP. It is derived from the isentropic relation:

```
q = (gamma / (gamma-1)) * P_static * isentropic_term
```

Step 5: Mach number

Mach is derived directly from the pressure ratio, not from velocity:

```
M = sqrt( (2 / (gamma-1)) * isentropic_term )
```

Step 6: Static temperature

The thermocouple reads stagnation temperature T0. Static temperature is recovered using the isentropic relation:

```
T_static = T0 / (1 + (gamma-1)/2 * M^2)
```

Step 7: Static density (ideal gas law)

Density is computed using **static** pressure and **static** temperature (both freestream quantities):

```
rho = P_static / (R_air * T_static)
```

Step 8: Speed of sound

Based on static temperature (the local freestream condition):

```
a = sqrt(gamma * R_air * T_static)
```

Step 9: Freestream velocity

```
U_inf = M * a
```

Step 10: Dynamic viscosity (Sutherland's law)

Temperature-dependent viscosity replaces the fixed-constant approximation:

```
mu = mu_ref * (T_static / T_ref)^(3/2) * (T_ref + S) / (T_static + S)
```

Where `mu_ref = 1.716e-5 Pa*s`, `T_ref = 273.15 K`, `S = 110.4 K`.

Step 11: Reynolds number

```
Re = rho * U_inf * L / mu
```

Where `L = C_inches * 0.0254` (reference chord converted to meters).

Physical Constants

Constant	Value	Units	Description
PSI_TO_PA	6894.75729	Pa/psi	Pressure conversion
C_TO_K	273.15	K	Celsius to Kelvin offset
R_AIR	287.058	J/(kg*K)	Specific gas constant for air
GAMMA	1.4	--	Ratio of specific heats
MU_REF	1.716e-5	Pa*s	Sutherland reference viscosity
T_REF	273.15	K	Sutherland reference temperature
S_SUTH	110.4	K	Sutherland constant for air

Compressible vs. Incompressible Comparison

Quantity	Old (Incompressible)	New (Compressible Isentropic)
q	q = dP directly	q = (gamma/(gamma-1)) * P_static * [(P0/Ps)^((gamma-1)/gamma) - 1]
Mach	M = U/a (derived last)	M = sqrt((2/(gamma-1)) * isentropic_term) (derived first)
Density	rho = P0 / (R * T0)	rho = P_static / (R * T_static)
Temperature	T = T0 (thermocouple direct)	T_static = T0 / (1 + (gamma-1)/2 * M^2)
Velocity	U = sqrt(2*q/rho)	U = M * a
Viscosity	mu = 1.81e-5 (constant)	Sutherland's law (temperature-dependent)

Output: `TunnelConditions` containing `Q, Q_mks, P_tot, P_static, T0, T (static), rho, Mach, a, U_inf, Re` -- all time-series vectors.

Stage 8: Aerodynamic Coefficients

Source: `coefficients.py` -> `calc_aero_coeffs()`

Aerodynamic forces and moments are non-dimensionalized using dynamic pressure and reference geometry. Since both the forces and dynamic pressure are time-series, each sample produces an instantaneous coefficient value.

Coefficient Definitions

Coefficient	Formula	Reference Length	Description
C_L	Lift / (Q * S)	-	Lift coefficient
C_D	Drag / (Q * S)	-	Drag coefficient
C_Y	Side / (Q * S)	-	Side force coefficient
C_l (roll)	Roll / (Q * S * b)	span	Rolling moment coefficient
C_m (pitch)	Pitch / (Q * S * C)	chord	Pitching moment coefficient
C_n (yaw)	Yaw / (Q * S * b)	span	Yawing moment coefficient

Where:

- **Q** = Dynamic pressure (time-series, in consistent units with forces)
- **S** = Reference area
- **C** = Reference chord (MAC) — used for pitching moment
- **b** = Reference span — used for rolling and yawing moments

A floor of `1e-10` is applied to `Q * S` to prevent division by zero at wind-off conditions.

Pressure Coefficients

If pressure port data is available (channels named `P001`, `P002`, etc.), pressure coefficients are computed:

```
Cp_i = (P_on_i - P_off_i) / Q
```

Tare subtraction for pressure ports is done at the raw level before coefficient computation.

Output: `AeroCoefficients` containing `Cl, Cd, Cs, CRoll, CPitch, CYaw` — all **time-series vectors**. Each sample is a fully-reduced, physically meaningful coefficient value.

Stage 9: Steady-State Reduction

Source: `reduction.py` -> `reduce_steady_state()`

The final stage collapses each test point's coefficient time-series into a single representative value.

Mean Values (Steady-State)

For each test point `i`, the steady-state coefficient is the temporal mean:

```
CL_i = mean(CL(t))    over all samples in test point i
CD_i = mean(CD(t))
...
```

Standard Deviations (Flow Unsteadiness)

The within-point temporal standard deviation captures flow unsteadiness:

```
CL_std_i = std(CL(t))    over all samples in test point i
CD_std_i = std(CD(t))
...
```

This can be displayed as shaded bands on coefficient plots (mean +/- 1 sigma).

Grid Organization

Steady-state data is organized by alpha and beta:

1. **Compute mean alpha/beta** per point: `alpha_i = mean(alpha(t))`, `beta_i = mean(beta(t))`
2. **Round** to nearest 0.5 degrees for grouping: `alpha_int = round(alpha * 2) / 2`
3. **Sort** by alpha (primary) then beta (secondary) using `np.lexsort`
4. **Reshape** into 2D grid if `n_alpha * n_beta == n_points`:

   ```
   CL[n_alpha x n_beta]   — each row is an alpha sweep at constant beta
   ```

   Otherwise, data remains as sorted 1D arrays.

Output: `SteadyStateData` containing 2D (or 1D) arrays for all coefficients, their standard deviations, alpha, beta, and pressure coefficients.

Pressure Transducer Calibration

Source: `calibration.py` -> `read_pcf_file()`

The `.PCF` file stores calibration data for pressure transducers (differential pressure, total pressure, and any additional pressure ports).

Each transducer entry contains:

- **Transducer ID** (e.g., "220" for differential pressure, "690" for total pressure)
- **Calibration date**

- **Slope (positive):** Pressure = raw_voltage * slope (units: PSI/V typically)
- **Slope (negative):** For noise characterization
- **Excitation voltage**
- **Units** (typically PSI)

Usage: The slope converts raw transducer voltage to engineering pressure units. For tunnel conditions, `Pdiff * slope_220` gives dynamic pressure in PSI, and `Ptot * slope_690` gives total pressure in PSI.

Model Geometry

Source: `transforms.py` -> `Geometry` dataclass

The model geometry defines the reference values used for non-dimensionalization and moment transfers.

Parameter	Symbol	Description
MAC	C	Mean aerodynamic chord — reference length for pitching moment
Reference Area	S	Wing planform area — reference area for all coefficients
Reference Span	b	Wing span — reference length for rolling and yawing moments
MRC Shift	mshift	[x, y, z] offset from balance center to desired moment reference center

MRC Shift Convention

The MRC shift vector `[mshift_x, mshift_y, mshift_z]` defines the offset from the balance mechanical center to the desired aerodynamic moment reference center:

- **x:** Along body axis (positive forward)
- **y:** Lateral (positive to port)
- **z:** Vertical (positive down)

Multi-Geometry Support

Multiple named geometry definitions can be configured, each with its own MAC, span, reference area, and MRC. Individual test cases can be assigned different geometries (e.g., different wing configurations tested on the same balance). Cases are assigned geometry via right-click context menu, and re-reduction is triggered automatically.

Derived Aerodynamic Parameters

Source: `coefficients.py`

The following parameters are computed from the steady-state coefficient data:

Lift Curve Slope (C_L_alpha):

```
Linear least-squares fit of CL vs alpha in the specified range (default: -5 to
+10 deg)
C_L_alpha = slope of the fit (per degree)
```

Zero-Lift Angle of Attack (alpha_0):

```
alpha_0 = -intercept / slope    from the CL vs alpha linear fit
```

Drag Polar Coefficients:

```
Least-squares fit: CD = CD0 + K * CL^2
Returns CD0 (zero-lift drag) and K (induced drag factor)
```

Oswald Efficiency Factor:

```
e = 1 / (pi * AR * K)
where AR is the wing aspect ratio
```

Maximum Lift-to-Drag Ratio:

```
(L/D)_max = 1 / (2 * sqrt(CD0 * K))
CL at (L/D)_max = sqrt(CD0 / K)
```

Static Margin:

```
SM = -Cm_alpha / CL_alpha
Positive values indicate static longitudinal stability.
```

Graphical User Interface

Launch: `python streamlined.py`

Architecture

The GUI follows the Model-View-Controller (MVC) pattern:

- **Model** (`data_model.py`, `case.py`): Central data store, test case representation, plot configuration
- **Views** (`main_window.py`, `plot_panel.py`, `table_panel.py`, `data_panel.py`): UI components
- **Controller** (`data_controller.py`): Data processing logic, reduction orchestration

Dual Canvas Architecture

Two plot backends are supported:

- **pyqtgraph** `FastPlotCanvas` (preferred): GPU-accelerated, fast interactive plotting
- **matplotlib** `PlotCanvas` (fallback): Publication-quality static plots

Both implement the same API: `plot()`, `clear()`, `refresh()`, `set_labels()`, `toggle_grid()`, `add_legend()`, `remove_legend()`, `autoscale()`, `fill_between()`.

Features

Data Management:

- Load balance calibration (.vol) and pressure calibration (.PCF)
- Define multiple named geometries with per-case assignment
- Load data directories with automatic AirOn/AirOff file classification
- Save/load full configuration presets (JSON format with relative paths)

Interactive Plotting:

- Real-time coefficient plots with multiple case overlay
- Standard deviation shading (mean +/- sigma bands)
- Adjustable line width and marker size controls
- Right-click context menu: view time history, view FFT at nearest data point
- Alpha and beta multi-select filters with popup selection widgets
- Legend toggle, grid toggle, auto-range

Interactive Plot Export:

- Customizable legend labels, font sizes
- Adjustable line widths, marker sizes, and marker styles per trace
- Color picker per trace
- Theme selection: Light, Dark, Black, Transparent
- Axis label and limit customization
- Persistent settings memory across export sessions

Data Table:

- Full coefficient data with optional tunnel conditions columns
- Numeric sorting (not string-based)
- Per-case Excel sheets with summary headers

Plot Types:

Plot Type	X-Axis	Y-Axis
CL vs Alpha	alpha	C_L
CD vs Alpha	alpha	C_D

Plot Type	X-Axis	Y-Axis
CL vs CD	C_D	C_L
Cm vs Alpha	alpha	C_m
Cm vs CL	C_L	C_m
L/D vs Alpha	alpha	L/D
CY vs Alpha	alpha	C_Y
C_l (roll) vs Alpha	alpha	C_l
C_n (yaw) vs Alpha	alpha	C_n
Lateral vs Beta	beta	C_Y

Export Capabilities

Unified Export Dialog (File > Export, Ctrl+Shift+E):

Format	Extension	Description
CSV	.csv	Comma-separated, one file per case
Excel	.xlsx	Multi-sheet workbook, one sheet per case with summary headers
HDF5	.h5	Hierarchical structure with optional raw/reduced time-series
MAT	.mat	MATLAB-compatible, nested structs via scipy.io

HDF5 Structure (when extended data enabled):

```
/calibration/           – balance and pressure cal metadata
/geometry/              – reference values (MAC, span, area, MRC)
/<case_name>/
    averaged/           – steady-state coefficients and tunnel conditions
    air_on/point_0/     – raw air-on time-series per test point
    air_off/point_0/    – raw air-off time-series
    reduced/point_0/    – fully-reduced coefficient time-series
        coefficients/   – CL(t), CD(t), Cm(t), ...
        tunnel/         – Q(t), Mach(t), Re(t), ...
        elements/       – balance element forces (N1/N2/Y1/Y2 or
AftPitch/AftYaw/FwdPitch/FwdYaw)
```

Technical Specifications

Dependencies

Package	Version	Purpose
numpy	>= 1.20	Numerical arrays and linear algebra
scipy	>= 1.7	Interpolation, curve fitting, MAT export
pandas	>= 1.3	DataFrame operations, CSV/Excel export
matplotlib	>= 3.5	Fallback plotting backend
nptdms	>= 1.6	TDMS file reading
PyQt6	>= 6.4	GUI framework
pyqtgraph	>= 0.13	Fast interactive plotting (optional, preferred)
h5py	>= 3.0	HDF5 export (optional)

Supported Facilities

Facility	Code	Description
Subsonic Wind Tunnel	SWT	Full reduction (Q, T, P0, rho, U, Mach, Re)
Low-Speed Wind Tunnel	LSWT	Dynamic pressure only (atmospheric conditions)
Trisonic Tunnel	TST	Compressible flow (placeholder)

Unit Systems

System	Length	Area	Pressure
IPS	inches	sq inches	psi
FPS	feet	sq feet	psf
MKS	meters	sq meters	Pa
CGS	centimeters	sq centimeters	Pa

Geometry input units define how MAC, reference area, and MRC are interpreted. Output units control dimensional display of tunnel conditions. Aerodynamic coefficients are dimensionless and unaffected by unit selection.

Validation

The package has been validated against MATLAB reference outputs:

- **Coefficient Agreement**: Within machine precision (< 1e-10 difference)
- **Transformation Verification**: BRF and WRF forces match MATLAB exactly
- **Calibration Fidelity**: Polynomial coefficients identical to MATLAB `lstsq`
- **Tunnel Conditions**: Temperature, pressure, density, velocity, Mach, Reynolds all verified

---

Document Version: 1.2.4
Author: C. Fagley
Last Updated: March 2026